Dual-Arm Architecture Implementation Plan

> **For Claude:** REQUIRED SUB-SKILL: Use superpowers:executing-plans to implement this plan task-by-task.

Goal: Replace the Mamba+LLM gated fusion architecture with a dual-arm system (Mamba behavioral + BGE/FAISS content tower), combined at a reranker layer.

Architecture: Two independent arms — Mamba4Rec predicts from interaction sequences, Content Tower retrieves by semantic similarity. A reranker blends scores with implicit alpha based on query specificity. Graduation mechanism promotes new movies into Mamba's catalog.

Tech Stack: PyTorch, RecBole, sentence-transformers (BAAI/bge-large-en-v1.5), faiss-cpu, polars, aiohttp

Design doc: `docs/plans/2026-02-20-dual-arm-architecture-design.md`

Task 1: Strip Fusion from Mamba4Rec

Files: - Modify: `mamba4rec.py` - Test: `tests/test_mamba4rec.py` (create)

The model class `Mamba4RecFusion` gets stripped to pure `Mamba4Rec`. Remove all fusion imports, fusion attributes, phase management, alignment loss, and gate logic.

Step 1: Write the failing test

Create `tests/test_mamba4rec.py`:

```
"""Tests for pure Mamba4Rec (no fusion)."""
import pytest
import torch
from unittest.mock import MagicMock


def _make_config(hidden_size=64, num_layers=1):
    """Create a minimal mock config for Mamba4Rec."""
    config = MagicMock()
```

```python
    config.__getitem__ = lambda self, key: {
        "hidden_size": hidden_size,
        "num_layers": num_layers,
        "dropout_prob": 0.1,
        "loss_type": "CE",
        "d_state": 16,
        "d_conv": 4,
        "expand": 2,
        "MAX_ITEM_LIST_LENGTH": 50,
    }[key]
    config.__contains__ = lambda self, key: key in {
        "hidden_size", "num_layers", "dropout_prob", "loss_type",
        "d_state", "d_conv", "expand", "MAX_ITEM_LIST_LENGTH",
    }
    config.final_config_dict = {
        "hidden_size": hidden_size,
        "loss_type": "CE",
    }
    return config


def _make_dataset(n_items=100):
    """Create a minimal mock dataset."""
    dataset = MagicMock()
    dataset.num = MagicMock(return_value=n_items)
    return dataset


class TestMamba4Rec:
    def test_no_fusion_attributes(self):
        """Model should not have any fusion-related attributes."""
        from mamba4rec import Mamba4Rec
        model = Mamba4Rec(_make_config(), _make_dataset())
        assert not hasattr(model, "fusion")
        assert not hasattr(model, "llm_projection")
        assert not hasattr(model, "_current_phase")
        assert not hasattr(model, "use_llm_fusion")

    def test_forward_shape(self):
        """Forward pass should return (B, hidden_size)."""
        from mamba4rec import Mamba4Rec
        model = Mamba4Rec(_make_config(), _make_dataset())
        model.eval()
        item_seq = torch.randint(0, 100, (4, 10))
        item_seq_len = torch.tensor([10, 8, 5, 3])
        with torch.no_grad():
            output = model.forward(item_seq, item_seq_len)
        assert output.shape == (4, 64)

    def test_full_sort_predict_shape(self):
        """full_sort_predict should return (B, n_items) scores."""
        from mamba4rec import Mamba4Rec
        model = Mamba4Rec(_make_config(), _make_dataset())
        model.eval()
```

```
        interaction = {
            "item_id_list": torch.randint(0, 100, (4, 10)),
            "item_length": torch.tensor([10, 8, 5, 3]),
        }
        with torch.no_grad():
            scores = model.full_sort_predict(interaction)
        assert scores.shape[0] == 4

    def test_no_set_training_phase(self):
        """set_training_phase should not exist."""
        from mamba4rec import Mamba4Rec
        model = Mamba4Rec(_make_config(), _make_dataset())
        assert not hasattr(model, "set_training_phase")
```

Step 2: Run test to verify it fails

Run: `pytest tests/test_mamba4rec.py -v` Expected: FAIL — `Mamba4Rec` is an alias to `Mamba4RecFusion` which has fusion attributes.

Step 3: Strip mamba4rec.py

Remove these from `mamba4rec.py`: 1. **Imports** (lines ~9-11): Remove `from fusion import ...` and `from llm_projection import ...` 2. **Class rename**: `Mamba4RecFusion` → `Mamba4Rec` 3. **`__init__`**: Remove `use_llm_fusion` config reads, `llm_projection` creation, `fusion` creation, `_current_phase` attribute 4. **Remove methods**: `_apply_fusion`, `calculate_loss_with_alignment`, `set_training_phase`, `_unfreeze_all`, `_freeze_for_phase2`, `_configure_phase3`, `get_fusion_weights` 5. **Simplify `calculate_loss`**: Remove the `if self._current_phase >= 2` fusion branch 6. **Simplify `predict`/`full_sort_predict`**: Remove fusion interaction prep 7. **Remove alias**: Delete `Mamba4Rec = Mamba4RecFusion` at bottom

The resulting class should have: - `__init__`, `_init_weights`, `forward`, `calculate_loss`, `predict`, `full_sort_predict`, `get_trainable_params`, `get_frozen_params`

Step 4: Run test to verify it passes

Run: `pytest tests/test_mamba4rec.py -v` Expected: 4 PASS

Step 5: Commit

```
git add mamba4rec.py tests/test_mamba4rec.py
git commit -m "Strip fusion from Mamba4Rec to pure sequential model"
```

Task 2: Delete Fusion and Projection Modules

Files: - Delete: `fusion.py` - Delete: `llm_projection.py` - Modify: `inference.py` (remove fusion imports to prevent import errors)

Step 1: Delete the files

```
rm fusion.py llm_projection.py
```

Step 2: Fix imports in inference.py temporarily

Remove these lines from `inference.py`: - `from mamba4rec import Mamba4RecFusion` - `from llm_projection import LLMProjectionWithAlignment`

Add a comment at top: `# TODO: Rewrite as dual-arm orchestrator (Task 14)`

For now, just make it importable without errors. The full rewrite comes in Task 14.

Step 3: Verify no import errors

Run: `python -c "import mamba4rec; print('OK')"` Expected: `OK`

Step 4: Run existing tests

Run: `pytest tests/ -v` Expected: All pipeline tests pass. Mamba tests from Task 1 pass.

Step 5: Commit

```
git add -u
git commit -m "Remove fusion.py and llm_projection.py"
```

Task 3: Simplify Training to Single Phase

Files: - Rename: `train_phases.py` → `train.py` (overwrite existing `run.py`-based train) - Delete: old `run.py` - Test: `tests/test_train.py` (create)

Step 1: Write the failing test

Create `tests/test_train.py`:

```
"""Tests for single-phase training module."""
import pytest
from unittest.mock import MagicMock, patch
```

```python
class TestTrainModule:
    def test_no_ewc_class(self):
        """EWCRegularizer should not exist."""
        import train
        assert not hasattr(train, "EWCRegularizer")

    def test_no_phase2_phase3(self):
        """Multi-phase functions should not exist."""
        import train
        assert not hasattr(train, "run_phase2")
        assert not hasattr(train, "run_phase3")

    def test_has_train_function(self):
        """Should have a train() function."""
        import train
        assert callable(getattr(train, "train_mamba", None))

    def test_has_expand_embeddings(self):
        """Should have a function to expand item embeddings for
        retraining."""
        import train
        assert callable(getattr(train, "expand_item_embeddings", None))
```

Step 2: Run test to verify it fails

Run: `pytest tests/test_train.py -v` Expected: FAIL — current `train.py` doesn't exist (there's `train_phases.py` and `run.py`)

Step 3: Create train.py

Write `train.py` — single-phase Mamba training with retraining support:

```python
"""Single-phase Mamba4Rec training with retraining support."""
import sys
import os
import argparse
import logging
from pathlib import Path

import torch
from torch.optim import Adam
from torch.optim.lr_scheduler import CosineAnnealingLR

from recbole.config import Config
from recbole.data import create_dataset, data_preparation
from recbole.trainer import Trainer
from recbole.utils import init_seed, init_logger

from mamba4rec import Mamba4Rec

# PyTorch 2.6+ compat
```

```python
_original_torch_load = torch.load
def _patched_torch_load(*args, **kwargs):
    kwargs.setdefault("weights_only", False)
    return _original_torch_load(*args, **kwargs)
torch.load = _patched_torch_load

logger = logging.getLogger(__name__)


def expand_item_embeddings(model, new_num_items):
    """Expand the item embedding table for retraining with graduated
        movies.

    Args:
        model: Mamba4Rec model with existing embeddings.
        new_num_items: New total number of items (must be >= current).

    Returns:
        Model with expanded embedding table (new rows randomly
        initialized).
    """
    old_emb = model.item_embedding
    old_num, emb_dim = old_emb.weight.shape

    if new_num_items <= old_num:
        return model

    new_emb = torch.nn.Embedding(new_num_items, emb_dim, padding_idx=0)
    with torch.no_grad():
        new_emb.weight[:old_num] = old_emb.weight
        torch.nn.init.normal_(new_emb.weight[old_num:], mean=0.0,
        std=0.02)

    model.item_embedding = new_emb
    logger.info(f"Expanded embeddings: {old_num} -> {new_num_items}
        items")
    return model


def train_mamba(config_path, save_dir="checkpoints", checkpoint=None):
    """Train Mamba4Rec from scratch or continue from checkpoint.

    Args:
        config_path: Path to YAML config file.
        save_dir: Directory to save the trained model.
        checkpoint: Optional path to existing checkpoint for retraining.

    Returns:
        Tuple of (model, test_result).
    """
    save_dir = Path(save_dir)
    save_dir.mkdir(parents=True, exist_ok=True)

    config = Config(model=Mamba4Rec, config_file_list=[config_path])
    init_seed(config["seed"], config["reproducibility"])
```

```python
    init_logger(config)

    dataset = create_dataset(config)
    train_data, valid_data, test_data = data_preparation(config, dataset)

    model = Mamba4Rec(config, dataset).to(config["device"])

    if checkpoint:
        state = torch.load(checkpoint, map_location=config["device"])
        model_state = state.get("state_dict", state)
        # Handle embedding size mismatch for retraining
        old_num = model_state["item_embedding.weight"].shape[0]
        new_num = model.item_embedding.weight.shape[0]
        if new_num > old_num:
            model = expand_item_embeddings(model, new_num)
            model.load_state_dict(model_state, strict=False)
        else:
            model.load_state_dict(model_state)
        logger.info(f"Loaded checkpoint from {checkpoint}")

    total = sum(p.numel() for p in model.parameters())
    trainable = sum(p.numel() for p in model.parameters() if
        p.requires_grad)
    logger.info(f"Parameters: {total:,} total, {trainable:,} trainable")

    trainer = Trainer(config, model)
    best_valid_score, best_valid_result = trainer.fit(
        train_data, valid_data, show_progress=True
    )

    test_result = trainer.evaluate(test_data)
    logger.info(f"Best valid: {best_valid_result}")
    logger.info(f"Test result: {test_result}")

    save_path = save_dir / "mamba4rec.pt"
    torch.save({"state_dict": model.state_dict(), "config": config},
        save_path)
    logger.info(f"Saved model to {save_path}")

    return model, test_result


def main():
    parser = argparse.ArgumentParser(description="Train Mamba4Rec")
    parser.add_argument("--config", default="config_ml32m.yaml")
    parser.add_argument("--save-dir", default="checkpoints")
    parser.add_argument("--checkpoint", default=None, help="Resume from
        checkpoint")
    args = parser.parse_args()

    train_mamba(args.config, args.save_dir, args.checkpoint)
```

```
if __name__ == "__main__":
    main()
```

Delete old `run.py`:

```
rm run.py
```

Delete old `train_phases.py`:

```
rm train_phases.py
```

Step 4: Run test to verify it passes

Run: `pytest tests/test_train.py -v` Expected: 4 PASS

Step 5: Commit

```
git add train.py tests/test_train.py
git rm run.py train_phases.py
git commit -m "Replace multi-phase training with single-phase train.py"
```

Task 4: Add Dependencies

Files: - Modify: `pyproject.toml`

Step 1: Add faiss-cpu and update sentence-transformers

In `pyproject.toml`, update the dependencies:

Add to core dependencies:

```
"faiss-cpu >= 1.7.0",
```

Step 2: Install

Run: `uv sync` Expected: Installs faiss-cpu successfully.

Step 3: Verify

Run: `python -c "import faiss; print(f'FAISS {faiss.__version__}')"`
Expected: Prints FAISS version.

Step 4: Commit

```
git add pyproject.toml uv.lock
git commit -m "Add faiss-cpu dependency"
```

Task 5: Swap LLM Encoder to BGE

Files: - Modify: `llm_encoder.py` - Test: `tests/test_llm_encoder.py` (create)

The encoder swaps from `all-MiniLM-L6-v2` (384-dim) to `BAAI/bge-large-en-v1.5` (1024-dim). Keep `IntentParser` unchanged. Simplify encoding interface.

Step 1: Write the failing test

Create `tests/test_llm_encoder.py`:

```python
"""Tests for BGE encoder."""
import pytest


class TestLLMEncoder:
    def test_default_model_is_bge(self):
        from llm_encoder import LLMEncoder
        enc = LLMEncoder.__new__(LLMEncoder)
        enc._model_name = "BAAI/bge-large-en-v1.5"
        assert enc._model_name == "BAAI/bge-large-en-v1.5"

    def test_embedding_dim_constant(self):
        from llm_encoder import EMBEDDING_DIM
        assert EMBEDDING_DIM == 1024

    def test_encode_query_exists(self):
        """Should have encode_query for user search queries."""
        from llm_encoder import LLMEncoder
        assert hasattr(LLMEncoder, "encode_query")

    def test_encode_plot_exists(self):
        """Should have encode_plot for movie plot texts."""
        from llm_encoder import LLMEncoder
        assert hasattr(LLMEncoder, "encode_plot")


class TestIntentParser:
    def test_mood_keywords(self):
        from llm_encoder import IntentParser
        parser = IntentParser()
        result = parser.parse("I want something scary and exciting")
        assert "scary" in result["mood"]
        assert "exciting" in result["mood"]
```

```
    def test_genre_keywords(self):
        from llm_encoder import IntentParser
        parser = IntentParser()
        result = parser.parse("show me a good comedy")
        assert "comedy" in result["genre"]

    def test_era_keywords(self):
        from llm_encoder import IntentParser
        parser = IntentParser()
        result = parser.parse("something from the 90s")
        assert "90s" in result["era"]
```

Step 2: Run test to verify it fails

Run: `pytest tests/test_llm_encoder.py -v` Expected: FAIL — no `EMBEDDING_DIM` constant, no `encode_query` / `encode_plot` methods.

Step 3: Rewrite llm_encoder.py

Key changes: 1. Default model → `BAAI/bge-large-en-v1.5` 2. Add `EMBEDDING_DIM = 1024` constant 3. Rename `encode_mood` → `encode_query` (for user search queries) 4. Rename `encode_movie_description` → `encode_plot` (for movie plots) 5. BGE queries need prefix `"Represent this sentence: "` for optimal results 6. Keep `IntentParser` as-is 7. Remove `encode_user_mood` module-level function

```
"""BGE encoder for plot embeddings and user queries.

Uses BAAI/bge-large-en-v1.5 (1024-dim) for both plot encoding
and query encoding — must be the same model/space.
"""
import hashlib
from functools import lru_cache
from typing import Dict, List, Optional, Union

import torch

EMBEDDING_DIM = 1024
DEFAULT_MODEL = "BAAI/bge-large-en-v1.5"


class LLMEncoder:
    """Encodes text into BGE embedding space.

    Same model is used for plots (documents) and queries (search).
    BGE recommends prefixing queries with an instruction for retrieval.
    """

    def __init__(
        self,
        model_name: str = DEFAULT_MODEL,
```

```
        device: str = None,
        cache_size: int = 10000,
    ):
        self._model_name = model_name
        self._device = device or ("cuda" if torch.cuda.is_available()
        else "cpu")
        self._model = None
        self._cache: Dict[str, torch.Tensor] = {}
        self._cache_order: List[str] = []
        self._cache_size = cache_size

    @property
    def model(self):
        if self._model is None:
            from sentence_transformers import SentenceTransformer
            self._model = SentenceTransformer(self._model_name,
        device=self._device)
        return self._model

    @property
    def embedding_dim(self) -> int:
        return EMBEDDING_DIM

    def _get_cache_key(self, text: str) -> str:
        return hashlib.md5(text.encode()).hexdigest()

    def _add_to_cache(self, key: str, embedding: torch.Tensor):
        if key in self._cache:
            return
        if len(self._cache) >= self._cache_size:
            oldest = self._cache_order.pop(0)
            self._cache.pop(oldest, None)
        self._cache[key] = embedding
        self._cache_order.append(key)

    def encode(
        self,
        texts: Union[str, List[str]],
        return_tensors: bool = True,
        use_cache: bool = True,
    ) -> torch.Tensor:
        """Encode raw text(s) into BGE embeddings.

        For retrieval tasks, use encode_query or encode_plot instead.
        """
        if isinstance(texts, str):
            texts = [texts]

        results = [None] * len(texts)
        to_encode = []
        to_encode_idx = []

        if use_cache:
            for i, text in enumerate(texts):
                key = self._get_cache_key(text)
```

```
                if key in self._cache:
                    results[i] = self._cache[key]
                else:
                    to_encode.append(text)
                    to_encode_idx.append(i)
        else:
            to_encode = texts
            to_encode_idx = list(range(len(texts)))

        if to_encode:
            embeddings = self.model.encode(
                to_encode,
                convert_to_tensor=return_tensors,
                show_progress_bar=len(to_encode) > 100,
                normalize_embeddings=True,
            )
            if return_tensors:
                embeddings = embeddings.to(self._device)

            for idx, emb_idx in enumerate(to_encode_idx):
                emb = embeddings[idx] if len(to_encode) > 1 else
     embeddings[0] if len(to_encode) == 1 else embeddings
                if len(to_encode) == 1 and embeddings.dim() == 1:
                    emb = embeddings
                else:
                    emb = embeddings[idx]
                results[emb_idx] = emb
                if use_cache:

     self._add_to_cache(self._get_cache_key(to_encode[idx]), emb)

        stacked = torch.stack(results)
        return stacked

    def encode_query(self, query: str) -> torch.Tensor:
        """Encode a user search query.

        BGE recommends prefixing queries with an instruction
        for asymmetric retrieval tasks.
        """
        prefixed = f"Represent this sentence for searching relevant
         passages: {query}"
        return self.encode(prefixed, use_cache=False).squeeze(0)

    def encode_plot(self, plot_text: str) -> torch.Tensor:
        """Encode a movie plot/overview (document side)."""
        return self.encode(plot_text).squeeze(0)

    def encode_plots_batch(
        self, texts: List[str], batch_size: int = 64
    ) -> torch.Tensor:
        """Encode multiple plot texts in batches."""
        all_embeddings = []
        for i in range(0, len(texts), batch_size):
            batch = texts[i : i + batch_size]
```

```
            embs = self.encode(batch, use_cache=False)
            all_embeddings.append(embs)
        return torch.cat(all_embeddings, dim=0)


class IntentParser:
    """Parse user messages for mood, genre, era, and constraints."""

    MOOD_KEYWORDS = {
        "cozy": ["cozy", "warm", "comfort", "heartwarming", "feel-good",
        "feelgood"],
        "exciting": ["exciting", "thrilling", "intense", "edge of seat",
        "adrenaline"],
        "scary": ["scary", "horror", "frightening", "terrifying",
        "creepy", "spooky"],
        "funny": ["funny", "comedy", "hilarious", "laugh", "humorous",
        "witty"],
        "sad": ["sad", "emotional", "cry", "tearjerker", "melancholy",
        "bittersweet"],
        "romantic": ["romantic", "love", "romance", "love story",
        "passionate"],
        "thoughtful": ["thoughtful", "deep", "philosophical", "thought-
        provoking", "cerebral", "mind-bending"],
        "relaxing": ["relaxing", "chill", "calm", "peaceful",
        "soothing", "easy-going"],
    }

    GENRE_KEYWORDS = {
        "action": ["action", "fight", "martial arts", "battle"],
        "comedy": ["comedy", "funny", "sitcom"],
        "drama": ["drama", "dramatic"],
        "horror": ["horror", "zombie", "slasher"],
        "scifi": ["sci-fi", "science fiction", "space", "futuristic",
        "cyberpunk"],
        "fantasy": ["fantasy", "magic", "wizard", "dragon"],
        "thriller": ["thriller", "suspense", "mystery", "detective"],
        "romance": ["romance", "romantic", "love story"],
        "documentary": ["documentary", "true story", "real life"],
        "animation": ["animated", "animation", "cartoon", "anime",
        "pixar"],
    }

    ERA_KEYWORDS = {
        "classic": ["classic", "old", "golden age", "vintage"],
        "80s": ["80s", "1980s", "eighties"],
        "90s": ["90s", "1990s", "nineties"],
        "2000s": ["2000s", "early 2000s"],
        "2010s": ["2010s", "twenty-tens"],
        "recent": ["recent", "new", "latest", "modern", "contemporary"],
    }

    def parse(self, message: str) -> dict:
        text = message.lower()
        return {
            "raw_text": message,
            "mood": self._extract_matches(text, self.MOOD_KEYWORDS),
```

```
            "genre": self._extract_matches(text, self.GENRE_KEYWORDS),
            "era": self._extract_matches(text, self.ERA_KEYWORDS),
            "constraints": self._extract_constraints(text),
        }

    def _extract_matches(self, text: str, keyword_dict: dict) -> list:
        matches = []
        for category, keywords in keyword_dict.items():
            if any(kw in text for kw in keywords):
                matches.append(category)
        return matches

    def _extract_constraints(self, text: str) -> dict:
        constraints = {}
        if any(w in text for w in ["short", "quick", "under 90", "90
        min"]):
            constraints["max_duration"] = 90
        if any(w in text for w in ["long", "epic", "over 2 hours"]):
            constraints["min_duration"] = 120
        if any(w in text for w in ["family", "kids", "children", "family-
        friendly"]):
            constraints["family_friendly"] = True
        if any(w in text for w in ["mature", "adult", "r-rated",
        "graphic"]):
            constraints["mature_only"] = True
        return constraints
```

Step 4: Run test to verify it passes

Run: `pytest tests/test_llm_encoder.py -v` Expected: All PASS

Step 5: Commit

```
git add llm_encoder.py tests/test_llm_encoder.py
git commit -m "Swap LLM encoder to BAAI/bge-large-en-v1.5 (1024-dim)"
```

Task 6: Create pipeline/filter_plots.py

Files: - Create: `pipeline/filter_plots.py` - Test: `tests/pipeline/test_filter_plots.py` (create)

Filters `wiki_plots_raw.parquet` (~193k articles) to movies/TV only using Wikidata SPARQL.

Step 1: Write the failing test

Create `tests/pipeline/test_filter_plots.py`:

```python
"""Tests for Wikidata movie/TV filtering."""
import pytest
import polars as pl


class TestFilterPlots:
    def test_filter_to_movies_removes_non_movies(self):
        """Given raw plots and a wikidata result, filter correctly."""
        from pipeline.filter_plots import filter_plots_to_movies_tv

        raw = pl.DataFrame({
            "wiki_title": ["The Matrix", "Harry Potter (novel)",
        "Breaking Bad"],
            "plot_text": ["Neo discovers...", "Harry is a wizard...",
        "Walter White..."],
        })
        wikidata_titles = {"The Matrix", "Breaking Bad"}

        result = filter_plots_to_movies_tv(raw, wikidata_titles)
        assert len(result) == 2
        titles = result["wiki_title"].to_list()
        assert "The Matrix" in titles
        assert "Breaking Bad" in titles
        assert "Harry Potter (novel)" not in titles

    def test_filter_preserves_columns(self):
        """Output should have same columns as input."""
        from pipeline.filter_plots import filter_plots_to_movies_tv

        raw = pl.DataFrame({
            "wiki_title": ["The Matrix"],
            "plot_text": ["Neo discovers..."],
        })
        result = filter_plots_to_movies_tv(raw, {"The Matrix"})
        assert "wiki_title" in result.columns
        assert "plot_text" in result.columns
```

Step 2: Run test to verify it fails

Run: `pytest tests/pipeline/test_filter_plots.py -v` Expected: FAIL — module doesn't exist.

Step 3: Implement pipeline/filter_plots.py

```python
"""Filter wiki_plots_raw.parquet to movies/TV only via Wikidata
        SPARQL."""
import json
import time
from pathlib import Path
from typing import Set

import polars as pl
import requests
```

```python
from pipeline.download import DATA_DIR, REPORTS_DIR

WIKIDATA_SPARQL_URL = "https://query.wikidata.org/sparql"

# Instance-of types for movies and TV
SPARQL_QUERY = """
SELECT DISTINCT ?articleTitle WHERE {{
  ?article schema:isPartOf <https://en.wikipedia.org/> ;
           schema:name ?articleTitle ;
           schema:about ?item .
  ?item wdt:P31 ?instance .
  VALUES ?instance {{
    wd:Q11424      wd:Q24856      wd:Q5398426
    wd:Q21191270   wd:Q24862      wd:Q506240
    wd:Q1261214    wd:Q63952888   wd:Q220898
  }}
}}
"""


def fetch_movie_tv_titles() -> Set[str]:
    """Fetch all Wikipedia article titles that are movies or TV from
        Wikidata."""
    headers = {
        "Accept": "application/sparql-results+json",
        "User-Agent": "Mamba4RecFusion/1.0 (research; polite)",
    }

    for attempt in range(5):
        try:
            resp = requests.get(
                WIKIDATA_SPARQL_URL,
                params={"query": SPARQL_QUERY},
                headers=headers,
                timeout=300,
            )
            if resp.status_code == 429:
                wait = 30 * (attempt + 1)
                print(f"  Rate limited, waiting {wait}s...")
                time.sleep(wait)
                continue
            resp.raise_for_status()
            data = resp.json()
            titles = set()
            for binding in data["results"]["bindings"]:
                titles.add(binding["articleTitle"]["value"])
            return titles
        except Exception as e:
            if attempt < 4:
                print(f"  Attempt {attempt + 1} failed: {e},
        retrying...")
                time.sleep(10 * (attempt + 1))
                continue
            raise
```

```python
    return set()


def filter_plots_to_movies_tv(
    raw_plots: pl.DataFrame, movie_tv_titles: Set[str]
) -> pl.DataFrame:
    """Filter raw plots DataFrame to only movies/TV titles."""
    return
        raw_plots.filter(pl.col("wiki_title").is_in(list(movie_tv_titles)))


def main():
    """Run the full filter pipeline."""
    raw_path = DATA_DIR / "wiki_plots_raw.parquet"
    output_path = DATA_DIR / "wiki_plots_movies_tv.parquet"

    if not raw_path.exists():
        print(f"ERROR: {raw_path} not found. Run the extraction pipeline
        first.")
        return

    raw = pl.read_parquet(raw_path)
    print(f"Raw plots: {len(raw)} articles")

    print("Fetching movie/TV titles from Wikidata...")
    movie_tv_titles = fetch_movie_tv_titles()
    print(f"Wikidata returned {len(movie_tv_titles)} movie/TV titles")

    filtered = filter_plots_to_movies_tv(raw, movie_tv_titles)
    print(f"After filtering: {len(filtered)} movie/TV articles")

    filtered.write_parquet(output_path)
    print(f"Wrote {output_path}")

    # Report
    report = {
        "raw_count": len(raw),
        "wikidata_titles": len(movie_tv_titles),
        "filtered_count": len(filtered),
        "removed_count": len(raw) - len(filtered),
    }
    report_path = REPORTS_DIR / "filter_plots_report.json"
    with open(report_path, "w") as f:
        json.dump(report, f, indent=2)
    print(f"Report: {report}")


if __name__ == "__main__":
    main()
```

Step 4: Run test to verify it passes

Run: `pytest tests/pipeline/test_filter_plots.py -v` Expected: 2 PASS

Step 5: Commit

```
git add pipeline/filter_plots.py tests/pipeline/test_filter_plots.py
git commit -m "Add Wikidata SPARQL filter for movie/TV plots"
```

Task 7: Create pipeline/encode_plots.py

Files: - Create: `pipeline/encode_plots.py` - Test: `tests/pipeline/test_encode_plots.py` (create)

Encodes all plot texts with BGE, prepends metadata, builds a FAISS IndexFlatIP index.

Step 1: Write the failing test

Create `tests/pipeline/test_encode_plots.py`:

```
"""Tests for plot encoding and FAISS index building."""
import pytest
import numpy as np


class TestPrependMetadata:
    def test_prepends_metadata(self):
        from pipeline.encode_plots import prepend_metadata
        result = prepend_metadata("Neo discovers...", "Film", "Action,
        Sci-Fi", 1999, "The Matrix")
        assert result == "Film. Action, Sci-Fi. 1999. The Matrix. Neo
        discovers..."

    def test_handles_missing_year(self):
        from pipeline.encode_plots import prepend_metadata
        result = prepend_metadata("A story...", "Film", "Drama", None,
        "Unknown")
        assert result == "Film. Drama. Unknown. A story..."

    def test_handles_missing_genre(self):
        from pipeline.encode_plots import prepend_metadata
        result = prepend_metadata("A story...", "Film", None, 2020,
        "Movie")
        assert result == "Film. 2020. Movie. A story..."


class TestBuildFaissIndex:
    def test_build_index_shape(self):
        from pipeline.encode_plots import build_faiss_index
        import faiss
        embeddings = np.random.randn(10, 1024).astype(np.float32)
        index = build_faiss_index(embeddings)
```

```
        assert index.ntotal == 10
        assert index.d == 1024

    def test_search_returns_correct_k(self):
        from pipeline.encode_plots import build_faiss_index
        embeddings = np.random.randn(20, 1024).astype(np.float32)
        # Normalize for inner product
        norms = np.linalg.norm(embeddings, axis=1, keepdims=True)
        embeddings = embeddings / norms
        index = build_faiss_index(embeddings)
        query = embeddings[0:1]
        scores, ids = index.search(query, 5)
        assert ids.shape == (1, 5)
        assert ids[0, 0] == 0  # closest to itself
```

Step 2: Run test to verify it fails

Run: `pytest tests/pipeline/test_encode_plots.py -v` Expected: FAIL — module doesn't exist.

Step 3: Implement pipeline/encode_plots.py

```
"""Encode movie plots with BGE and build FAISS index."""
import json
from pathlib import Path
from typing import Optional

import faiss
import numpy as np
import polars as pl
from tqdm import tqdm

from pipeline.download import DATA_DIR, REPORTS_DIR

EMBEDDING_DIM = 1024
DEFAULT_MODEL = "BAAI/bge-large-en-v1.5"


def prepend_metadata(
    plot_text: str,
    media_type: str = "Film",
    genres: Optional[str] = None,
    year: Optional[int] = None,
    title: Optional[str] = None,
) -> str:
    """Prepend structured metadata to plot text for embedding
        normalization."""
    parts = [media_type]
    if genres:
        parts.append(genres)
    if year is not None:
        parts.append(str(year))
    if title:
```

```
        parts.append(title)
    prefix = ". ".join(parts)
    return f"{prefix}. {plot_text}"


def build_faiss_index(embeddings: np.ndarray) -> faiss.IndexFlatIP:
    """Build a FAISS inner-product index from embeddings."""
    dim = embeddings.shape[1]
    index = faiss.IndexFlatIP(dim)
    # Normalize for cosine similarity via inner product
    faiss.normalize_L2(embeddings)
    index.add(embeddings)
    return index


def encode_and_build_index(
    plots_df: pl.DataFrame,
    model_name: str = DEFAULT_MODEL,
    batch_size: int = 64,
    index_path: Path = DATA_DIR / "plots.faiss",
    metadata_path: Path = DATA_DIR / "plots_metadata.parquet",
):
    """Encode all plots and build FAISS index.

    Args:
        plots_df: DataFrame with columns: wiki_title or title, plot_text,
                  and optionally genres, year, plot_source.
        model_name: Sentence transformer model name.
        batch_size: Encoding batch size.
        index_path: Where to save the FAISS index.
        metadata_path: Where to save the metadata mapping.
    """
    from sentence_transformers import SentenceTransformer

    model = SentenceTransformer(model_name)

    # Determine title column
    title_col = "title" if "title" in plots_df.columns else "wiki_title"

    # Prepare texts with metadata
    texts = []
    for row in tqdm(plots_df.iter_rows(named=True), total=len(plots_df),
        desc="Preparing texts"):
        title = row.get(title_col, "")
        genres = row.get("genres", None)
        year = row.get("year", None)
        plot_source = row.get("plot_source", "")
        media_type = "Film"  # Default; could detect from genres
        if genres and "Animation" in str(genres):
            media_type = "Animated Film"

        text = prepend_metadata(row["plot_text"], media_type, genres,
        year, title)
        texts.append(text)
```

```python
    # Encode in batches
    print(f"Encoding {len(texts)} plots with {model_name}...")
    all_embeddings = model.encode(
        texts,
        batch_size=batch_size,
        show_progress_bar=True,
        normalize_embeddings=True,
        convert_to_numpy=True,
    )
    all_embeddings = all_embeddings.astype(np.float32)
    print(f"Embeddings shape: {all_embeddings.shape}")

    # Build FAISS index
    index = faiss.IndexFlatIP(all_embeddings.shape[1])
    index.add(all_embeddings)
    print(f"FAISS index: {index.ntotal} vectors, {index.d} dimensions")

    # Save index
    index_path.parent.mkdir(parents=True, exist_ok=True)
    faiss.write_index(index, str(index_path))
    print(f"Saved FAISS index to {index_path}")

    # Save metadata (position → movie info mapping)
    metadata_cols = [c for c in plots_df.columns if c != "plot_text"]
    metadata_df =
        plots_df.select(metadata_cols).with_row_index("faiss_idx")
    metadata_df.write_parquet(metadata_path)
    print(f"Saved metadata to {metadata_path}")

    # Report
    report = {
        "total_encoded": len(texts),
        "embedding_dim": int(all_embeddings.shape[1]),
        "model": model_name,
        "index_path": str(index_path),
        "metadata_path": str(metadata_path),
    }
    report_path = REPORTS_DIR / "encode_plots_report.json"
    with open(report_path, "w") as f:
        json.dump(report, f, indent=2)
    print(f"Report: {report}")


def main():
    """Encode plots and build index from movie_plots.parquet."""
    plots_path = DATA_DIR / "movie_plots.parquet"
    if not plots_path.exists():
        print(f"ERROR: {plots_path} not found.")
        return

    plots_df = pl.read_parquet(plots_path)
    print(f"Loaded {len(plots_df)} plots")

    encode_and_build_index(plots_df)
```

```
if __name__ == "__main__":
    main()
```

Step 4: Run test to verify it passes

Run: `pytest tests/pipeline/test_encode_plots.py -v` Expected: 5 PASS

Step 5: Commit

```
git add pipeline/encode_plots.py tests/pipeline/test_encode_plots.py
git commit -m "Add BGE plot encoding and FAISS index builder"
```

Task 8: Create content_tower.py

Files: - Create: `content_tower.py` - Test: `tests/test_content_tower.py` (create)

FAISS index management, content retrieval, runtime movie addition.

Step 1: Write the failing test

Create `tests/test_content_tower.py`:

```
"""Tests for content tower."""
import pytest
import numpy as np
import faiss
import polars as pl
import tempfile
from pathlib import Path


def _make_test_index(n=50, dim=1024):
    """Create a test FAISS index with random normalized vectors."""
    vecs = np.random.randn(n, dim).astype(np.float32)
    faiss.normalize_L2(vecs)
    index = faiss.IndexFlatIP(dim)
    index.add(vecs)
    return index, vecs


def _make_test_metadata(n=50):
    """Create test metadata DataFrame."""
    return pl.DataFrame({
        "faiss_idx": list(range(n)),
        "movieId": list(range(1, n + 1)),
        "title": [f"Movie {i}" for i in range(1, n + 1)],
```

```
    })


class TestContentTower:
    def test_search_by_query(self):
        from content_tower import ContentTower
        index, vecs = _make_test_index(50)
        metadata = _make_test_metadata(50)
        tower = ContentTower(index, metadata)

        query_vec = vecs[0]  # should match itself
        results = tower.search(query_embedding=query_vec, top_k=5)
        assert len(results) == 5
        assert results[0]["movieId"] == 1  # closest to itself

    def test_search_by_profile(self):
        from content_tower import ContentTower
        index, vecs = _make_test_index(50)
        metadata = _make_test_metadata(50)
        tower = ContentTower(index, metadata)

        profile = vecs[10]
        results = tower.search(profile_embedding=profile, top_k=5)
        assert len(results) == 5
        assert results[0]["movieId"] == 11

    def test_search_blended(self):
        from content_tower import ContentTower
        index, vecs = _make_test_index(50)
        metadata = _make_test_metadata(50)
        tower = ContentTower(index, metadata)

        results = tower.search(
            query_embedding=vecs[0],
            profile_embedding=vecs[10],
            alpha=0.5,
            top_k=5,
        )
        assert len(results) == 5

    def test_search_returns_scores(self):
        from content_tower import ContentTower
        index, vecs = _make_test_index(50)
        metadata = _make_test_metadata(50)
        tower = ContentTower(index, metadata)

        results = tower.search(query_embedding=vecs[0], top_k=3)
        for r in results:
            assert "score" in r
            assert "movieId" in r

    def test_add_movie(self):
        from content_tower import ContentTower
        index, vecs = _make_test_index(50)
```

```
        metadata = _make_test_metadata(50)
        tower = ContentTower(index, metadata)

        new_vec = np.random.randn(1024).astype(np.float32)
        new_vec /= np.linalg.norm(new_vec)
        tower.add_movie(new_vec, {"movieId": 999, "title": "New Movie"})
        assert tower.index.ntotal == 51

    def test_save_and_load(self):
        from content_tower import ContentTower
        index, vecs = _make_test_index(10)
        metadata = _make_test_metadata(10)
        tower = ContentTower(index, metadata)

        with tempfile.TemporaryDirectory() as tmpdir:
            idx_path = Path(tmpdir) / "test.faiss"
            meta_path = Path(tmpdir) / "test_meta.parquet"
            tower.save(idx_path, meta_path)

            loaded = ContentTower.load(idx_path, meta_path)
            assert loaded.index.ntotal == 10
```

Step 2: Run test to verify it fails

Run: `pytest tests/test_content_tower.py -v` Expected: FAIL — module doesn't exist.

Step 3: Implement content_tower.py

```
"""Content Tower — FAISS-based semantic movie retrieval."""
from pathlib import Path
from typing import Dict, List, Optional

import faiss
import numpy as np
import polars as pl


class ContentTower:
    """Manages a FAISS index of movie plot embeddings for content
        retrieval."""

    def __init__(self, index: faiss.IndexFlatIP, metadata: pl.DataFrame):
        self.index = index
        self.metadata = metadata

    @classmethod
    def load(cls, index_path: Path, metadata_path: Path) ->
        "ContentTower":
        """Load a saved FAISS index and metadata."""
        index = faiss.read_index(str(index_path))
        metadata = pl.read_parquet(metadata_path)
        return cls(index, metadata)
```

```python
    def save(self, index_path: Path, metadata_path: Path):
        """Save the FAISS index and metadata to disk."""
        index_path.parent.mkdir(parents=True, exist_ok=True)
        faiss.write_index(self.index, str(index_path))
        self.metadata.write_parquet(metadata_path)

    def search(
        self,
        query_embedding: Optional[np.ndarray] = None,
        profile_embedding: Optional[np.ndarray] = None,
        alpha: float = 0.5,
        top_k: int = 50,
    ) -> List[Dict]:
        """Search the content index.

        Args:
            query_embedding: From user's text query (1024-dim,
        normalized).
            profile_embedding: From user's taste profile (1024-dim,
        normalized).
            alpha: Blend weight — 1.0 = query only, 0.0 = profile only.
            top_k: Number of results to return.

        Returns:
            List of dicts with movieId, title, score, and faiss_idx.
        """
        if query_embedding is None and profile_embedding is None:
            return []

        if query_embedding is not None and profile_embedding is not None:
            search_vec = alpha * query_embedding + (1 - alpha) *
        profile_embedding
        elif query_embedding is not None:
            search_vec = query_embedding
        else:
            search_vec = profile_embedding

        # Normalize the blended vector
        norm = np.linalg.norm(search_vec)
        if norm > 0:
            search_vec = search_vec / norm

        search_vec = search_vec.reshape(1, -1).astype(np.float32)
        scores, indices = self.index.search(search_vec, top_k)

        results = []
        for i in range(top_k):
            idx = int(indices[0, i])
            if idx < 0:
                continue
            row = self.metadata.filter(pl.col("faiss_idx") == idx)
            if row.is_empty():
                continue
            entry = row.to_dicts()[0]
```

```
            entry["score"] = float(scores[0, i])
            results.append(entry)

        return results

    def get_embedding(self, faiss_idx: int) -> np.ndarray:
        """Retrieve the stored embedding for a given FAISS index
        position."""
        return self.index.reconstruct(faiss_idx)

    def add_movie(self, embedding: np.ndarray, metadata_row: Dict):
        """Add a new movie to the index at runtime.

        Args:
            embedding: Normalized 1024-dim embedding.
            metadata_row: Dict with at least movieId and title.
        """
        new_idx = self.index.ntotal
        vec = embedding.reshape(1, -1).astype(np.float32)
        self.index.add(vec)

        metadata_row["faiss_idx"] = new_idx
        new_row = pl.DataFrame([metadata_row])
        self.metadata = pl.concat(
            [self.metadata, new_row], how="diagonal_relaxed"
        )

    def movie_id_to_faiss_idx(self, movie_id: int) -> Optional[int]:
        """Look up the FAISS index position for a movieId."""
        row = self.metadata.filter(pl.col("movieId") == movie_id)
        if row.is_empty():
            return None
        return row["faiss_idx"][0]

    def get_movie_embedding(self, movie_id: int) -> Optional[np.ndarray]:
        """Get the plot embedding for a movie by its movieId."""
        idx = self.movie_id_to_faiss_idx(movie_id)
        if idx is None:
            return None
        return self.get_embedding(idx)
```

Step 4: Run test to verify it passes

Run: `pytest tests/test_content_tower.py -v` Expected: 6 PASS

Step 5: Commit

```
git add content_tower.py tests/test_content_tower.py
git commit -m "Add ContentTower with FAISS index and retrieval"
```

Task 9: Update embedding_store.py for Content Profiles

Files: - Modify: `embedding_store.py` - Test: `tests/test_embedding_store.py` (create)

Update profile dimensions from 64 → 1024. Change the profile update method to accept plot embeddings weighted by ratings instead of mood+feedback.

Step 1: Write the failing test

Create `tests/test_embedding_store.py`:

```
"""Tests for updated embedding store with 1024-dim content profiles."""
import pytest
import numpy as np
import torch


class TestContentProfile:
    def test_update_profile_with_rating(self):
        """Profile should update from plot embedding + rating weight."""
        from embedding_store import InMemoryProfileStore
        store = InMemoryProfileStore(dim=1024)
        plot_emb = np.random.randn(1024).astype(np.float32)
        plot_emb /= np.linalg.norm(plot_emb)

        store.update_with_rating(user_id=1, plot_embedding=plot_emb,
         rating_weight=2.0)
        profile = store.get(1)
        assert profile is not None
        assert profile.vector.shape == (1024,)

    def test_profile_ema_update(self):
        """Successive updates should blend via EMA."""
        from embedding_store import InMemoryProfileStore
        store = InMemoryProfileStore(dim=1024, decay=0.9, base_lr=0.1)

        emb1 = np.zeros(1024, dtype=np.float32)
        emb1[0] = 1.0
        emb2 = np.zeros(1024, dtype=np.float32)
        emb2[1] = 1.0

        store.update_with_rating(user_id=1, plot_embedding=emb1,
         rating_weight=1.0)
        profile1 = store.get(1).vector.copy()

        store.update_with_rating(user_id=1, plot_embedding=emb2,
         rating_weight=1.0)
        profile2 = store.get(1).vector
```

```
        # After second update, profile should have components in both
        dimensions
        assert profile2[0] > 0
        assert profile2[1] > 0

    def test_negative_rating_weight(self):
        """Negative rating weight should push profile away."""
        from embedding_store import InMemoryProfileStore
        store = InMemoryProfileStore(dim=1024)

        emb = np.zeros(1024, dtype=np.float32)
        emb[0] = 1.0

        # First: positive interaction
        store.update_with_rating(user_id=1, plot_embedding=emb,
        rating_weight=2.0)
        p1 = store.get(1).vector[0]

        # Second: negative interaction with same embedding
        store.update_with_rating(user_id=1, plot_embedding=emb,
        rating_weight=-2.0)
        p2 = store.get(1).vector[0]

        assert p2 < p1  # should have decreased


class TestEmbeddingManager:
    def test_prepare_content_profile_dict(self):
        """Should produce a dict with CONTENT_PROFILE key."""
        from embedding_store import EmbeddingManager
        mgr = EmbeddingManager(hidden_size=1024)

        # Manually set a profile
        from embedding_store import InMemoryProfileStore, ProfileEntry
        import time
        mgr.profile_store.set(1, ProfileEntry(
            vector=np.random.randn(1024).astype(np.float32),
            interaction_count=5,
            last_updated=time.time(),
            created_at=time.time(),
            user_id=1,
        ))

        result = mgr.get_profile_vector(1)
        assert result is not None
        assert result.shape == (1024,)
```

Step 2: Run test to verify it fails

Run: `pytest tests/test_embedding_store.py -v` Expected: FAIL — `InMemoryProfileStore` doesn't accept `dim` parameter, no `update_with_rating` method.

Step 3: Update embedding_store.py

Key changes: 1. Add `dim` parameter to `InMemoryProfileStore.__init__` (default 1024) 2. Add `update_with_rating(user_id, plot_embedding, rating_weight)` method 3. Update `EmbeddingManager.__init__` default `hidden_size` to 1024 4. Keep `update_with_feedback` for backward compat but mark as legacy 5. Keep all mood store logic (still needed for session mood tracking)

In `InMemoryProfileStore.__init__`, add `dim=1024` parameter:

```
def __init__(self, decay=0.95, base_lr=0.1, dim=1024):
    self._store = {}
    self._mood_history = {}
    self._decay = decay
    self._base_lr = base_lr
    self._dim = dim
```

Add `update_with_rating` method:

```
def update_with_rating(self, user_id: int, plot_embedding: np.ndarray,
         rating_weight: float):
    """Update user profile from a plot embedding weighted by rating.

    rating_weight = rating - 3 for 5-star scale (so 5→+2, 1→-2)
    rating_weight = +1 for right swipe, -1 for left swipe
    """
    existing = self.get(user_id)
    weighted = rating_weight * plot_embedding

    if existing is None:
        norm = np.linalg.norm(weighted)
        vector = weighted / norm if norm > 0 else weighted
        self.set(user_id, ProfileEntry(
            vector=vector,
            interaction_count=1,
            last_updated=time.time(),
            created_at=time.time(),
            user_id=user_id,
        ))
    else:
        lr = self._base_lr / (1 + 0.01 * existing.interaction_count)
        new_vector = self._decay * existing.vector + lr * weighted
        norm = np.linalg.norm(new_vector)
        if norm > 0:
            new_vector = new_vector / norm
        self.set(user_id, ProfileEntry(
            vector=new_vector,
            interaction_count=existing.interaction_count + 1,
            last_updated=time.time(),
            created_at=existing.created_at,
```

```
            user_id=user_id,
        ))
```

Step 4: Run test to verify it passes

Run: `pytest tests/test_embedding_store.py -v` Expected: 4 PASS

Step 5: Commit

```
git add embedding_store.py tests/test_embedding_store.py
git commit -m "Update embedding store for 1024-dim content profiles"
```

Task 10: Create reranker.py

Files: - Create: `reranker.py` - Test: `tests/test_reranker.py` (create)

Score normalization, implicit alpha, single-user and group ranking.

Step 1: Write the failing test

Create `tests/test_reranker.py`:

```
"""Tests for reranker module."""
import pytest
import numpy as np


class TestScoreNormalization:
    def test_min_max_normalize(self):
        from reranker import min_max_normalize
        scores = {"a": 10.0, "b": 20.0, "c": 30.0}
        normed = min_max_normalize(scores)
        assert normed["a"] == pytest.approx(0.0)
        assert normed["b"] == pytest.approx(0.5)
        assert normed["c"] == pytest.approx(1.0)

    def test_min_max_single_value(self):
        from reranker import min_max_normalize
        scores = {"a": 5.0}
        normed = min_max_normalize(scores)
        assert normed["a"] == pytest.approx(1.0)


class TestComputeAlpha:
    def test_no_query_no_profile(self):
        from reranker import compute_alpha
        alpha = compute_alpha(query_text=None, has_profile=False)
        assert alpha == 0.0
```

```python
    def test_no_query_with_profile(self):
        from reranker import compute_alpha
        alpha = compute_alpha(query_text=None, has_profile=True,
        home_feed_alpha=0.2)
        assert alpha == pytest.approx(0.2)

    def test_short_query(self):
        from reranker import compute_alpha
        alpha = compute_alpha(query_text="something fun",
        has_profile=True)
        assert 0.3 <= alpha <= 0.9

    def test_long_specific_query(self):
        from reranker import compute_alpha
        alpha = compute_alpha(
            query_text="a dark psychological thriller set in the 90s
        about a detective",
            has_profile=True,
        )
        assert alpha > 0.5  # should lean toward content


class TestReranker:
    def test_blend_scores(self):
        from reranker import Reranker
        rr = Reranker()
        mamba_scores = {1: 0.9, 2: 0.5, 3: 0.1}
        content_scores = {1: 0.2, 2: 0.8, 4: 0.7}
        result = rr.blend(mamba_scores, content_scores, alpha=0.5,
        top_k=3)
        assert len(result) == 3
        # Each result should have movie_id and score
        assert all("movie_id" in r and "score" in r for r in result)

    def test_content_only_for_new_movies(self):
        from reranker import Reranker
        rr = Reranker()
        mamba_scores = {}
        content_scores = {1: 0.9, 2: 0.7, 3: 0.5}
        result = rr.blend(mamba_scores, content_scores, alpha=0.8,
        top_k=3)
        assert len(result) == 3
        assert result[0]["movie_id"] == 1

    def test_mamba_only_for_no_plot(self):
        from reranker import Reranker
        rr = Reranker()
        mamba_scores = {1: 0.9, 2: 0.7}
        content_scores = {}
        result = rr.blend(mamba_scores, content_scores, alpha=0.5,
        top_k=2)
        assert len(result) == 2
```

```
class TestGroupRanking:
    def test_group_penalizes_disagreement(self):
        from reranker import Reranker
        rr = Reranker()
        # Movie A: user1 loves (0.9), user2 hates (0.1) → high std
        # Movie B: both like (0.6, 0.7) → low std
        per_user_scores = {
            "A": [0.9, 0.1],
            "B": [0.6, 0.7],
        }
        result = rr.rank_group(per_user_scores, fairness_lambda=0.5,
        top_k=2)
        # B should rank higher than A despite lower mean
        assert result[0]["movie_id"] == "B"
```

Step 2: Run test to verify it fails

Run: `pytest tests/test_reranker.py -v` Expected: FAIL — module doesn't exist.

Step 3: Implement reranker.py

```
"""Reranker — blends Mamba and Content Tower scores."""
import math
from typing import Dict, List, Optional

import numpy as np


def min_max_normalize(scores: Dict, epsilon: float = 1e-8) -> Dict:
    """Min-max normalize a dict of scores to [0, 1]."""
    if not scores:
        return {}
    values = list(scores.values())
    lo, hi = min(values), max(values)
    spread = hi - lo
    if spread < epsilon:
        return {k: 1.0 for k in scores}
    return {k: (v - lo) / spread for k, v in scores.items()}


def compute_alpha(
    query_text: Optional[str] = None,
    has_profile: bool = False,
    home_feed_alpha: float = 0.2,
    alpha_min: float = 0.3,
    alpha_max: float = 0.9,
) -> float:
    """Compute blend alpha from query specificity.

    Returns:
        Alpha in [0, 1] where higher = trust content more.
    """
    if query_text is None:
```

```
        if has_profile:
            return home_feed_alpha
        return 0.0

    words = query_text.split()
    word_count = len(words)

    # Simple heuristic: more words → more specific → higher alpha
    # Sigmoid mapping: 2 words → ~0.4, 5 words → ~0.6, 10+ words → ~0.8
    raw = 0.3 * word_count - 1.0
    alpha = 1.0 / (1.0 + math.exp(-raw))

    return max(alpha_min, min(alpha_max, alpha))


class Reranker:
    """Blends scores from Mamba (behavioral) and Content Tower
        (semantic)."""

    def blend(
        self,
        mamba_scores: Dict,
        content_scores: Dict,
        alpha: float,
        top_k: int = 10,
    ) -> List[Dict]:
        """Blend Mamba and content scores.

        Args:
            mamba_scores: {movie_id: raw_score} from Mamba arm.
            content_scores: {movie_id: cosine_sim} from content arm.
            alpha: Blend weight (1.0 = content only, 0.0 = mamba only).
            top_k: Number of results.

        Returns:
            Sorted list of {movie_id, score, mamba_score, content_score}.
        """
        # Normalize mamba scores
        mamba_normed = min_max_normalize(mamba_scores)

        # Content scores are already cosine similarity [0, 1]
        all_ids = set(mamba_normed.keys()) | set(content_scores.keys())

        scored = []
        for mid in all_ids:
            c_score = content_scores.get(mid, 0.0)
            m_score = mamba_normed.get(mid, 0.0)

            has_content = mid in content_scores
            has_mamba = mid in mamba_normed

            if has_content and has_mamba:
                final = alpha * c_score + (1 - alpha) * m_score
            elif has_content:
```

```
                final = alpha * c_score
            else:
                final = (1 - alpha) * m_score

            scored.append({
                "movie_id": mid,
                "score": final,
                "mamba_score": m_score if has_mamba else None,
                "content_score": c_score if has_content else None,
            })

        scored.sort(key=lambda x: -x["score"])
        return scored[:top_k]

    def rank_group(
        self,
        per_user_scores: Dict[str, List[float]],
        fairness_lambda: float = 0.5,
        top_k: int = 10,
    ) -> List[Dict]:
        """Rank movies for a group using fairness-weighted aggregation.

        Args:
            per_user_scores: {movie_id: [user1_score, user2_score, ...]}.
            fairness_lambda: Penalty weight for score disagreement.
            top_k: Number of results.

        Returns:
            Sorted list of {movie_id, score, mean, std}.
        """
        scored = []
        for mid, user_scores in per_user_scores.items():
            arr = np.array(user_scores)
            mean = float(np.mean(arr))
            std = float(np.std(arr))
            group_score = mean - fairness_lambda * std
            scored.append({
                "movie_id": mid,
                "score": group_score,
                "mean": mean,
                "std": std,
            })

        scored.sort(key=lambda x: -x["score"])
        return scored[:top_k]
```

Step 4: Run test to verify it passes

Run: `pytest tests/test_reranker.py -v` Expected: All PASS

Step 5: Commit

```
git add reranker.py tests/test_reranker.py
git commit -m "Add reranker with alpha blending and group ranking"
```

Task 11: Create graduation.py

Files: - Create: `graduation.py` - Test: `tests/test_graduation.py` (create)

Interaction counting, graduation queueing, retraining triggers.

Step 1: Write the failing test

Create `tests/test_graduation.py`:

```
"""Tests for graduation manager."""
import pytest
import json
import tempfile
from pathlib import Path


class TestGraduationManager:
    def test_record_interaction(self):
        from graduation import GraduationManager
        with tempfile.TemporaryDirectory() as tmpdir:
            gm = GraduationManager(
                queue_path=Path(tmpdir) / "queue.json",
                graduation_threshold=3,
                mamba_catalog=set(),
            )
            gm.record_interaction("tt001")
            gm.record_interaction("tt001")
            assert gm.get_interaction_count("tt001") == 2

    def test_graduation_on_threshold(self):
        from graduation import GraduationManager
        with tempfile.TemporaryDirectory() as tmpdir:
            gm = GraduationManager(
                queue_path=Path(tmpdir) / "queue.json",
                graduation_threshold=3,
                mamba_catalog=set(),
            )
            gm.record_interaction("tt001")
            gm.record_interaction("tt001")
            graduated = gm.record_interaction("tt001")
            assert graduated is True
            assert "tt001" in gm.get_pending_graduations()

    def test_no_graduation_for_mamba_movies(self):
        from graduation import GraduationManager
```

```
        with tempfile.TemporaryDirectory() as tmpdir:
            gm = GraduationManager(
                queue_path=Path(tmpdir) / "queue.json",
                graduation_threshold=3,
                mamba_catalog={"tt001"},
            )
            for _ in range(5):
                gm.record_interaction("tt001")
            assert "tt001" not in gm.get_pending_graduations()

    def test_persistence(self):
        from graduation import GraduationManager
        with tempfile.TemporaryDirectory() as tmpdir:
            queue_path = Path(tmpdir) / "queue.json"
            gm1 = GraduationManager(
                queue_path=queue_path,
                graduation_threshold=2,
                mamba_catalog=set(),
            )
            gm1.record_interaction("tt001")
            gm1.record_interaction("tt001")
            gm1.save()

            gm2 = GraduationManager(
                queue_path=queue_path,
                graduation_threshold=2,
                mamba_catalog=set(),
            )
            assert "tt001" in gm2.get_pending_graduations()

    def test_threshold_trigger(self):
        from graduation import GraduationManager
        with tempfile.TemporaryDirectory() as tmpdir:
            gm = GraduationManager(
                queue_path=Path(tmpdir) / "queue.json",
                graduation_threshold=1,
                mamba_catalog=set(),
                retrain_on_graduation_count=3,
            )
            gm.record_interaction("tt001")
            gm.record_interaction("tt002")
            gm.record_interaction("tt003")
            assert gm.should_retrain_by_threshold() is True

    def test_mark_retrained(self):
        from graduation import GraduationManager
        with tempfile.TemporaryDirectory() as tmpdir:
            gm = GraduationManager(
                queue_path=Path(tmpdir) / "queue.json",
                graduation_threshold=1,
                mamba_catalog=set(),
            )
            gm.record_interaction("tt001")
            gm.mark_retrained(["tt001"], batch_id="batch_001")
```

```
            assert "tt001" not in gm.get_pending_graduations()
            assert len(gm.get_completed()) == 1
```

Step 2: Run test to verify it fails

Run: `pytest tests/test_graduation.py -v` Expected: FAIL — module doesn't exist.

Step 3: Implement graduation.py

```
"""Graduation manager — tracks new movie interactions and retraining
        triggers."""
import json
import time
from pathlib import Path
from typing import Dict, List, Optional, Set


class GraduationManager:
    """Manages the lifecycle of new movies graduating into Mamba's
        catalog."""

    def __init__(
        self,
        queue_path: Path,
        graduation_threshold: int = 50,
        mamba_catalog: Optional[Set[str]] = None,
        retrain_on_graduation_count: int = 100,
    ):
        self._queue_path = queue_path
        self._threshold = graduation_threshold
        self._mamba_catalog = mamba_catalog or set()
        self._retrain_count = retrain_on_graduation_count

        self._interaction_counts: Dict[str, int] = {}
        self._pending: Dict[str, dict] = {}
        self._completed: List[dict] = []

        self._load()

    def _load(self):
        """Load state from disk if available."""
        if not self._queue_path.exists():
            return
        with open(self._queue_path) as f:
            data = json.load(f)
        for item in data.get("pending", []):
            self._pending[item["movie_id"]] = item
            self._interaction_counts[item["movie_id"]] =
        item.get("interaction_count", self._threshold)
        self._completed = data.get("completed", [])

    def save(self):
```

```
        """Persist state to disk."""
        self._queue_path.parent.mkdir(parents=True, exist_ok=True)
        data = {
            "pending": list(self._pending.values()),
            "completed": self._completed,
        }
        with open(self._queue_path, "w") as f:
            json.dump(data, f, indent=2)

    def record_interaction(self, movie_id: str) -> bool:
        """Record an interaction with a movie.

        Returns True if the movie just graduated (crossed threshold).
        """
        if movie_id in self._mamba_catalog:
            return False
        if movie_id in self._pending:
            self._interaction_counts[movie_id] =
         self._interaction_counts.get(movie_id, 0) + 1
            self._pending[movie_id]["interaction_count"] =
         self._interaction_counts[movie_id]
            return False

        self._interaction_counts[movie_id] =
         self._interaction_counts.get(movie_id, 0) + 1

        if self._interaction_counts[movie_id] >= self._threshold:
            self._pending[movie_id] = {
                "movie_id": movie_id,
                "graduated_at": time.strftime("%Y-%m-%d"),
                "interaction_count": self._interaction_counts[movie_id],
            }
            return True
        return False

    def get_interaction_count(self, movie_id: str) -> int:
        return self._interaction_counts.get(movie_id, 0)

    def get_pending_graduations(self) -> List[str]:
        return list(self._pending.keys())

    def get_completed(self) -> List[dict]:
        return list(self._completed)

    def should_retrain_by_threshold(self) -> bool:
        return len(self._pending) >= self._retrain_count

    def mark_retrained(self, movie_ids: List[str], batch_id: str):
        """Mark movies as retrained and move from pending to
        completed."""
        for mid in movie_ids:
            if mid in self._pending:
                entry = self._pending.pop(mid)
                entry["retrained_at"] = time.strftime("%Y-%m-%d")
                entry["batch"] = batch_id
```

```
                self._completed.append(entry)
                self._mamba_catalog.add(mid)

    def trigger_retrain(self, reason: str = "manual") ->
        Optional[List[str]]:
        """Get the list of movies to include in retraining.

        Returns None if no movies are pending.
        """
        pending = self.get_pending_graduations()
        if not pending:
            return None
        return pending
```

Step 4: Run test to verify it passes

Run: `pytest tests/test_graduation.py -v` Expected: 6 PASS

Step 5: Commit

```
git add graduation.py tests/test_graduation.py
git commit -m "Add graduation manager with interaction tracking and
        retraining triggers"
```

Task 12: Create chat_provider.py

Files: - Create: `chat_provider.py` - Test: `tests/test_chat_provider.py` (create)

Provider-agnostic chatbot LLM interface.

Step 1: Write the failing test

Create `tests/test_chat_provider.py`:

```
"""Tests for chat provider interface."""
import pytest


class TestChatProvider:
    def test_abstract_interface(self):
        from chat_provider import ChatProvider
        with pytest.raises(TypeError):
            ChatProvider()  # abstract, can't instantiate

    def test_format_recommendations_prompt(self):
        from chat_provider import format_prompt
        recs = [
            {"title": "The Matrix", "year": 1999, "genres": "Action, Sci-
        Fi", "plot_snippet": "A hacker discovers..."},
```

```
            {"title": "Inception", "year": 2010, "genres": "Sci-Fi,
        Thriller", "plot_snippet": "A thief enters..."},
        ]
        prompt = format_prompt("something mind-bending", recs)
        assert "something mind-bending" in prompt
        assert "The Matrix" in prompt
        assert "Inception" in prompt

    def test_openai_provider_exists(self):
        from chat_provider import OpenAIChat
        assert hasattr(OpenAIChat, "generate")

    def test_claude_provider_exists(self):
        from chat_provider import ClaudeChat
        assert hasattr(ClaudeChat, "generate")
```

Step 2: Run test to verify it fails

Run: `pytest tests/test_chat_provider.py -v` Expected: FAIL — module doesn't exist.

Step 3: Implement chat_provider.py

```
"""Provider-agnostic chatbot LLM for formatting recommendations."""
from abc import ABC, abstractmethod
from typing import Dict, List, Optional


def format_prompt(query: str, recommendations: List[Dict], max_recs: int
        = 5) -> str:
    """Format the recommendation prompt for any LLM provider."""
    recs_text = ""
    for i, rec in enumerate(recommendations[:max_recs], 1):
        title = rec.get("title", "Unknown")
        year = rec.get("year", "")
        genres = rec.get("genres", "")
        snippet = rec.get("plot_snippet", "")
        line = f"{i}. {title}"
        if year:
            line += f" ({year})"
        if genres:
            line += f" - {genres}"
        if snippet:
            line += f" - {snippet}"
        recs_text += line + "\n"

    return (
        f'The user asked: "{query}"\n'
        f"Based on their taste profile and history, here are the top
        recommendations:\n"
        f"{recs_text}\n"
        f"Generate a natural, conversational response presenting these
        recommendations. "
```

```
        f"Explain briefly why each might appeal to the user based on
        their query."
    )


class ChatProvider(ABC):
    """Abstract base class for chat LLM providers."""

    @abstractmethod
    def generate(self, prompt: str) -> str:
        """Generate a response from the LLM."""
        ...


class OpenAIChat(ChatProvider):
    """OpenAI GPT chat provider."""

    def __init__(self, model: str = "gpt-4o", api_key: Optional[str] =
        None):
        self._model = model
        self._api_key = api_key

    def generate(self, prompt: str) -> str:
        import openai
        client = openai.OpenAI(api_key=self._api_key)
        response = client.chat.completions.create(
            model=self._model,
            messages=[
                {"role": "system", "content": "You are a friendly movie
        recommendation assistant."},
                {"role": "user", "content": prompt},
            ],
            max_tokens=500,
        )
        return response.choices[0].message.content


class ClaudeChat(ChatProvider):
    """Anthropic Claude chat provider."""

    def __init__(self, model: str = "claude-sonnet-4-20250514", api_key:
        Optional[str] = None):
        self._model = model
        self._api_key = api_key

    def generate(self, prompt: str) -> str:
        import anthropic
        client = anthropic.Anthropic(api_key=self._api_key)
        response = client.messages.create(
            model=self._model,
            max_tokens=500,
            system="You are a friendly movie recommendation assistant.",
            messages=[{"role": "user", "content": prompt}],
        )
        return response.content[0].text
```

```python
class GeminiChat(ChatProvider):
    """Google Gemini chat provider."""

    def __init__(self, model: str = "gemini-pro", api_key: Optional[str]
        = None):
        self._model = model
        self._api_key = api_key

    def generate(self, prompt: str) -> str:
        import google.generativeai as genai
        genai.configure(api_key=self._api_key)
        model = genai.GenerativeModel(self._model)
        response = model.generate_content(
            f"You are a friendly movie recommendation assistant.
        \n\n{prompt}"
        )
        return response.text


def create_provider(provider: str, model: Optional[str] = None, api_key:
        Optional[str] = None) -> ChatProvider:
    """Factory function to create a chat provider."""
    providers = {
        "openai": OpenAIChat,
        "claude": ClaudeChat,
        "gemini": GeminiChat,
    }
    cls = providers.get(provider)
    if cls is None:
        raise ValueError(f"Unknown provider: {provider}. Available:
        {list(providers.keys())}")
    kwargs = {}
    if model:
        kwargs["model"] = model
    if api_key:
        kwargs["api_key"] = api_key
    return cls(**kwargs)
```

Step 4: Run test to verify it passes

Run: `pytest tests/test_chat_provider.py -v` Expected: 4 PASS

Step 5: Commit

```bash
git add chat_provider.py tests/test_chat_provider.py
git commit -m "Add provider-agnostic chat LLM interface"
```

Task 13: Rewrite inference.py as Dual-Arm Orchestrator

Files: - Rewrite: `inference.py` - Test: `tests/test_inference.py` (create)

The new `DualArmEngine` orchestrates both arms, computes alpha, calls the reranker, handles single-user and group recommendations.

Step 1: Write the failing test

Create `tests/test_inference.py`:

```python
"""Tests for dual-arm inference engine."""
import pytest
import numpy as np
import faiss
import polars as pl
from unittest.mock import MagicMock, patch


def _mock_content_tower(n=50, dim=1024):
    """Create a mock ContentTower."""
    from content_tower import ContentTower
    vecs = np.random.randn(n, dim).astype(np.float32)
    faiss.normalize_L2(vecs)
    index = faiss.IndexFlatIP(dim)
    index.add(vecs)
    metadata = pl.DataFrame({
        "faiss_idx": list(range(n)),
        "movieId": list(range(1, n + 1)),
        "title": [f"Movie {i}" for i in range(1, n + 1)],
    })
    return ContentTower(index, metadata)


def _mock_mamba_model():
    """Create a mock Mamba model that returns random scores."""
    model = MagicMock()
    model.eval = MagicMock(return_value=model)
    return model


class TestDualArmEngine:
    def test_has_recommend_method(self):
        from inference import DualArmEngine
        assert hasattr(DualArmEngine, "recommend")

    def test_has_recommend_group_method(self):
        from inference import DualArmEngine
        assert hasattr(DualArmEngine, "recommend_group")
```

```
    def test_content_only_recommendation(self):
        """When no Mamba model, should still return content-based
        results."""
        from inference import DualArmEngine
        tower = _mock_content_tower()
        engine = DualArmEngine(content_tower=tower, mamba_model=None)

        query_vec = np.random.randn(1024).astype(np.float32)
        query_vec /= np.linalg.norm(query_vec)

        results = engine.recommend(
            query_embedding=query_vec,
            top_k=5,
        )
        assert len(results) == 5

    def test_result_has_required_fields(self):
        from inference import DualArmEngine
        tower = _mock_content_tower()
        engine = DualArmEngine(content_tower=tower, mamba_model=None)

        query_vec = np.random.randn(1024).astype(np.float32)
        query_vec /= np.linalg.norm(query_vec)

        results = engine.recommend(query_embedding=query_vec, top_k=3)
        for r in results:
            assert "movie_id" in r
            assert "score" in r
```

Step 2: Run test to verify it fails

Run: `pytest tests/test_inference.py -v` Expected: FAIL — `DualArmEngine` doesn't exist.

Step 3: Rewrite inference.py

```
"""Dual-arm recommendation engine.

Orchestrates Mamba (behavioral) and Content Tower (semantic) arms,
computes alpha, calls the reranker, handles single and group
        recommendations.
"""
from dataclasses import dataclass, field
from typing import Any, Dict, List, Optional, Set

import numpy as np
import torch

from content_tower import ContentTower
from embedding_store import EmbeddingManager
from llm_encoder import IntentParser, LLMEncoder
from reranker import Reranker, compute_alpha, min_max_normalize
```

```python
@dataclass
class RecommendationResult:
    recommendations: List[Dict]
    alpha: float
    parsed_intent: Optional[Dict] = None
    mode: str = "dual"  # "dual", "content_only", "mamba_only"


class DualArmEngine:
    """Orchestrates Mamba + Content Tower for recommendations."""

    def __init__(
        self,
        content_tower: Optional[ContentTower] = None,
        mamba_model: Optional[Any] = None,
        encoder: Optional[LLMEncoder] = None,
        embedding_manager: Optional[EmbeddingManager] = None,
        reranker: Optional[Reranker] = None,
        mamba_catalog: Optional[Set[int]] = None,
        device: str = "cpu",
        home_feed_alpha: float = 0.2,
        alpha_min: float = 0.3,
        alpha_max: float = 0.9,
    ):
        self.content_tower = content_tower
        self.mamba_model = mamba_model
        self.encoder = encoder or LLMEncoder()
        self.embedding_manager = embedding_manager or
         EmbeddingManager(hidden_size=1024)
        self.reranker = reranker or Reranker()
        self.intent_parser = IntentParser()
        self.mamba_catalog = mamba_catalog or set()
        self.device = device
        self.home_feed_alpha = home_feed_alpha
        self.alpha_min = alpha_min
        self.alpha_max = alpha_max

    def recommend(
        self,
        user_id: Optional[int] = None,
        item_history: Optional[List[int]] = None,
        query_text: Optional[str] = None,
        query_embedding: Optional[np.ndarray] = None,
        profile_embedding: Optional[np.ndarray] = None,
        top_k: int = 10,
        alpha_override: Optional[float] = None,
        exclude_history: bool = True,
    ) -> List[Dict]:
        """Generate recommendations using both arms.

        Args:
            user_id: User ID for Mamba and profile lookup.
            item_history: User's interaction history (item IDs) for
        Mamba.
```

```
        query_text: Natural language query (for chatbot mode).
        query_embedding: Pre-computed query embedding (optional).
        profile_embedding: Pre-computed profile embedding (optional).
        top_k: Number of recommendations to return.
        alpha_override: Override automatic alpha computation.
        exclude_history: Whether to exclude already-seen items.

    Returns:
        List of recommendation dicts with movie_id, score, etc.
    """
    parsed_intent = None

    # Encode query if text provided
    if query_text and query_embedding is None:
        query_embedding =
 self.encoder.encode_query(query_text).cpu().numpy()
        parsed_intent = self.intent_parser.parse(query_text)

    # Get profile if user_id provided
    if user_id is not None and profile_embedding is None:
        profile_vec =
 self.embedding_manager.get_profile_vector(user_id)
        if profile_vec is not None:
            profile_embedding = profile_vec.numpy() if
 isinstance(profile_vec, torch.Tensor) else profile_vec

    has_profile = profile_embedding is not None

    # Compute alpha
    if alpha_override is not None:
        alpha = alpha_override
    else:
        alpha = compute_alpha(
            query_text=query_text,
            has_profile=has_profile,
            home_feed_alpha=self.home_feed_alpha,
            alpha_min=self.alpha_min,
            alpha_max=self.alpha_max,
        )

    # Content arm
    content_scores = {}
    if self.content_tower and (query_embedding is not None or
 profile_embedding is not None):
        content_results = self.content_tower.search(
            query_embedding=query_embedding,
            profile_embedding=profile_embedding,
            alpha=alpha,
            top_k=50,
        )
        content_scores = {r["movieId"]: r["score"] for r in
 content_results}

    # Mamba arm
    mamba_scores = {}
```

```
        if self.mamba_model is not None and item_history:
            mamba_scores = self._get_mamba_scores(item_history, top_k=50)

        # Exclude history
        if exclude_history and item_history:
            history_set = set(item_history)
            content_scores = {k: v for k, v in content_scores.items() if
        k not in history_set}
            mamba_scores = {k: v for k, v in mamba_scores.items() if k
        not in history_set}

        # Determine mode
        if content_scores and mamba_scores:
            mode = "dual"
        elif content_scores:
            mode = "content_only"
        elif mamba_scores:
            mode = "mamba_only"
        else:
            return []

        # Rerank
        results = self.reranker.blend(mamba_scores, content_scores,
        alpha=alpha, top_k=top_k)

        return results

    def recommend_group(
        self,
        users: List[Dict],
        query_text: Optional[str] = None,
        top_k: int = 10,
        fairness_lambda: float = 0.5,
    ) -> List[Dict]:
        """Generate group recommendations.

        Args:
            users: List of user dicts, each with 'user_id' and optionally
                   'item_history' and 'profile_embedding'.
            query_text: Shared group query text.
            top_k: Number of recommendations.
            fairness_lambda: Penalty for score disagreement.

        Returns:
            List of recommendation dicts.
        """
        query_embedding = None
        if query_text:
            query_embedding =
        self.encoder.encode_query(query_text).cpu().numpy()

        # Gather candidates from all users
        all_candidates = set()

        for user in users:
```

```
            user_id = user.get("user_id")
            history = user.get("item_history", [])
            profile = user.get("profile_embedding")

            if self.mamba_model and history:
                mamba_top = self._get_mamba_scores(history, top_k=50)
                all_candidates |= set(mamba_top.keys())

            if self.content_tower and profile is not None:
                profile_results = self.content_tower.search(
                    profile_embedding=profile, top_k=30
                )
                all_candidates |= {r["movieId"] for r in profile_results}

        if self.content_tower and query_embedding is not None:
            query_results = self.content_tower.search(
                query_embedding=query_embedding, top_k=50
            )
            all_candidates |= {r["movieId"] for r in query_results}

        if not all_candidates:
            return []

        # Score each candidate per user
        per_movie_scores = {}
        for movie_id in all_candidates:
            user_scores = []
            for user in users:
                score = self._score_movie_for_user(movie_id, user,
        query_embedding)
                user_scores.append(score)
            per_movie_scores[movie_id] = user_scores

        return self.reranker.rank_group(
            per_movie_scores, fairness_lambda=fairness_lambda,
        top_k=top_k
        )

    def record_interaction(
        self,
        user_id: int,
        movie_id: int,
        rating_weight: float,
    ):
        """Record a user interaction and update their content profile.

        Args:
            user_id: User ID.
            movie_id: Movie they interacted with.
            rating_weight: Signal strength (+2 for 5-star, -2 for 1-star,
                           +1 for right swipe, -1 for left swipe).
        """
        if self.content_tower:
            plot_emb = self.content_tower.get_movie_embedding(movie_id)
            if plot_emb is not None:
```

```
            self.embedding_manager.profile_store.update_with_rating(
                user_id=user_id,
                plot_embedding=plot_emb,
                rating_weight=rating_weight,
            )

    def _get_mamba_scores(self, item_history: List[int], top_k: int =
        50) -> Dict[int, float]:
        """Get top-K scores from Mamba for a user's history."""
        if self.mamba_model is None:
            return {}

        with torch.no_grad():
            item_seq = torch.tensor([item_history], dtype=torch.long,
        device=self.device)
            item_seq_len = torch.tensor([len(item_history)],
        device=self.device)
            seq_output = self.mamba_model.forward(item_seq, item_seq_len)
            all_item_emb = self.mamba_model.item_embedding.weight
            scores = torch.matmul(seq_output, all_item_emb.T).squeeze(0)

        top_scores, top_indices = torch.topk(scores, min(top_k,
        len(scores)))
        return {
            int(idx): float(score)
            for idx, score in zip(top_indices.cpu(), top_scores.cpu())
        }

    def _score_movie_for_user(
        self,
        movie_id: int,
        user: Dict,
        query_embedding: Optional[np.ndarray],
    ) -> float:
        """Score a single movie for a single user (for group ranking)."""
        alpha = compute_alpha(
            query_text=None if query_embedding is None else "query",
            has_profile=user.get("profile_embedding") is not None,
            home_feed_alpha=self.home_feed_alpha,
        )

        content_score = 0.0
        if self.content_tower:
            movie_emb = self.content_tower.get_movie_embedding(movie_id)
            if movie_emb is not None:
                profile = user.get("profile_embedding")
                if profile is not None:
                    content_score = float(np.dot(movie_emb, profile))
                elif query_embedding is not None:
                    content_score = float(np.dot(movie_emb,
        query_embedding))

        mamba_score = 0.0
        if self.mamba_model and user.get("item_history"):
```

```
            full_scores = self._get_mamba_scores(user["item_history"],
        top_k=100)
            mamba_score = full_scores.get(movie_id, 0.0)

        return alpha * content_score + (1 - alpha) * mamba_score
```

Step 4: Run test to verify it passes

Run: `pytest tests/test_inference.py -v` Expected: 4 PASS

Step 5: Commit

```
git add inference.py tests/test_inference.py
git commit -m "Rewrite inference.py as dual-arm orchestrator"
```

Task 14: Update Config Files

Files: - Modify: `config_ml32m.yaml` - Delete: `config.yaml` (old ml-latest-small config) - Modify: `example_config.yaml`

Step 1: Update config_ml32m.yaml

Replace contents with the new dual-arm configuration from the design doc. Remove all fusion-related keys (`use_llm_fusion`, `llm_dim`, `fusion_dropout`, `vector_gate`, `phase2_*`, `phase3_*`, `ewc_lambda`). Add content tower, reranker, graduation, chatbot, and profile settings.

```yaml
# Mamba (behavioral arm)
dataset: ml-32m
hidden_size: 64
num_layers: 1
dropout_prob: 0.2
d_state: 32
d_conv: 4
expand: 2
loss_type: CE
epochs: 20
train_batch_size: 2048
learning_rate: 0.001
stopping_step: 5
MAX_ITEM_LIST_LENGTH: 50
eval_batch_size: 1024
metrics: ['Hit', 'NDCG', 'MRR']
valid_metric: 'NDCG@10'
rating_threshold: 3.5

# Content tower
embedding_model: BAAI/bge-large-en-v1.5
```

```yaml
embedding_dim: 1024
faiss_index_path: data/plots.faiss
plots_metadata_path: data/plots_metadata.parquet

# Reranker
alpha_mode: implicit
alpha_fixed: 0.5
alpha_min: 0.3
alpha_max: 0.9
home_feed_alpha: 0.2
group_fairness_lambda: 0.5

# Graduation
graduation_threshold: 50
retrain_trigger: all
retrain_schedule: weekly
retrain_on_graduation_count: 100
retraining_queue_path: data/retraining_queue.json

# Chatbot
chat_provider: openai
chat_model: gpt-4o

# User profiles
profile_decay: 0.95
profile_learning_rate: 0.1
profile_dim: 1024
mood_ttl: 1800
```

Step 2: Delete old config.yaml

```
rm config.yaml
```

Step 3: Update example_config.yaml

Copy the new config_ml32m.yaml contents into example_config.yaml with comments explaining each section.

Step 4: Verify no tests break

Run: `pytest tests/ -v` Expected: All PASS

Step 5: Commit

```
git add config_ml32m.yaml example_config.yaml
git rm config.yaml
git commit -m "Update configs for dual-arm architecture"
```

Task 15: Update pyproject.toml and Clean Up

Files: - Modify: `pyproject.toml` - Delete: `main.py` (6-line stub, unused)

Step 1: Update pyproject.toml

Add new optional dependency groups:

```
[project.optional-dependencies]
gpu = [
    "mamba-ssm >= 1.0.0; sys_platform == 'linux'",
    "causal-conv1d >= 1.0.0; sys_platform == 'linux'",
]
redis = ["redis >= 4.0.0"]
data = [
    "polars >= 1.0.0",
    "mwxml >= 0.3.3",
    "mwparserfromhell >= 0.6.5",
    "aiohttp >= 3.9.0",
    "tqdm >= 4.65.0",
    "requests >= 2.28.0",
]
chat = [
    "openai >= 1.0.0",
    "anthropic >= 0.18.0",
    "google-generativeai >= 0.3.0",
]
```

Add `faiss-cpu` to core deps, ensure `requests` is present (for filter_plots.py).

Step 2: Delete main.py

```
rm main.py
```

Step 3: Install and verify

Run: `uv sync` Expected: All deps install.

Step 4: Run full test suite

Run: `pytest tests/ -v` Expected: All PASS

Step 5: Commit

```
git add pyproject.toml uv.lock
git rm main.py
git commit -m "Update dependencies and clean up unused files"
```

Task 16: Final Integration Test

Files: - Create: `tests/test_integration.py`

End-to-end test that wires up ContentTower + Reranker + DualArmEngine with mock data.

Step 1: Write integration test

```python
"""Integration test for the full dual-arm pipeline."""
import pytest
import numpy as np
import faiss
import polars as pl

from content_tower import ContentTower
from reranker import Reranker, compute_alpha
from inference import DualArmEngine
from embedding_store import EmbeddingManager, InMemoryProfileStore
from graduation import GraduationManager
from llm_encoder import IntentParser
from chat_provider import format_prompt


def _build_test_content_tower(n=100, dim=1024):
    vecs = np.random.randn(n, dim).astype(np.float32)
    faiss.normalize_L2(vecs)
    index = faiss.IndexFlatIP(dim)
    index.add(vecs)
    metadata = pl.DataFrame({
        "faiss_idx": list(range(n)),
        "movieId": list(range(1, n + 1)),
        "title": [f"Movie {i}" for i in range(1, n + 1)],
        "genres": ["Action"] * 50 + ["Comedy"] * 50,
    })
    return ContentTower(index, metadata), vecs


class TestFullPipeline:
    def test_content_only_flow(self):
        """User with no history, just a query."""
        tower, vecs = _build_test_content_tower()
        engine = DualArmEngine(content_tower=tower, mamba_model=None)

        query_vec = np.random.randn(1024).astype(np.float32)
        query_vec /= np.linalg.norm(query_vec)

        results = engine.recommend(query_embedding=query_vec, top_k=10)
        assert len(results) == 10
        # Scores should be descending
        scores = [r["score"] for r in results]
        assert scores == sorted(scores, reverse=True)
```

```python
    def test_profile_builds_from_interactions(self):
        """Interactions should build a user profile."""
        tower, vecs = _build_test_content_tower()
        engine = DualArmEngine(content_tower=tower, mamba_model=None)

        # Simulate some positive interactions
        engine.record_interaction(user_id=1, movie_id=1,
         rating_weight=2.0)
        engine.record_interaction(user_id=1, movie_id=2,
         rating_weight=1.0)

        profile = engine.embedding_manager.profile_store.get(1)
        assert profile is not None
        assert profile.interaction_count == 2
        assert profile.vector.shape == (1024,)

    def test_graduation_flow(self):
        """New movie should graduate after enough interactions."""
        import tempfile
        from pathlib import Path

        with tempfile.TemporaryDirectory() as tmpdir:
            gm = GraduationManager(
                queue_path=Path(tmpdir) / "queue.json",
                graduation_threshold=5,
                mamba_catalog=set(),
            )

            for i in range(4):
                assert gm.record_interaction("tt_new_001") is False
            assert gm.record_interaction("tt_new_001") is True

            pending = gm.get_pending_graduations()
            assert "tt_new_001" in pending

            gm.mark_retrained(["tt_new_001"], batch_id="test_batch")
            assert len(gm.get_pending_graduations()) == 0
            assert len(gm.get_completed()) == 1

    def test_group_recommendation(self):
        """Group recommendation should work with multiple users."""
        tower, vecs = _build_test_content_tower()
        engine = DualArmEngine(content_tower=tower, mamba_model=None)

        users = [
            {"user_id": 1, "profile_embedding": vecs[0]},
            {"user_id": 2, "profile_embedding": vecs[50]},
        ]

        results = engine.recommend_group(
            users=users,
            query_text=None,
            top_k=5,
            fairness_lambda=0.5,
```

```
        )
        assert len(results) <= 5

    def test_intent_parser_integration(self):
        parser = IntentParser()
        result = parser.parse("I want a scary thriller from the 90s")
        assert "scary" in result["mood"]
        assert "thriller" in result["genre"]
        assert "90s" in result["era"]

    def test_format_prompt_integration(self):
        recs = [
            {"title": "Alien", "year": 1979, "genres": "Horror, Sci-Fi",
        "plot_snippet": "In space..."},
        ]
        prompt = format_prompt("something scary in space", recs)
        assert "Alien" in prompt
        assert "something scary in space" in prompt

    def test_alpha_computation_spectrum(self):
        """Alpha should increase with query specificity."""
        a_none = compute_alpha(query_text=None, has_profile=False)
        a_profile = compute_alpha(query_text=None, has_profile=True)
        a_short = compute_alpha(query_text="fun movies")
        a_long = compute_alpha(

        query_text="a dark psychological thriller set in a small town in
        the 1990s about a detective"
        )
        assert a_none == 0.0
        assert a_profile > 0.0
        assert a_long > a_short
```

Step 2: Run integration test

Run: `pytest tests/test_integration.py -v` Expected: All PASS

Step 3: Run full test suite

Run: `pytest tests/ -v` Expected: All tests pass.

Step 4: Commit

```
git add tests/test_integration.py
git commit -m "Add integration tests for dual-arm pipeline"
```

Task 17: Lint and Final Verification

Step 1: Run ruff

Run: `ruff check . --fix` Expected: No errors or all auto-fixed.

Step 2: Run full test suite

Run: `pytest tests/ -v` Expected: All PASS.

Step 3: Commit if any lint fixes

```
git add -u
git commit -m "Fix lint errors"
```

Execution Summary

Task	What	Creates/Modifies
1	Strip fusion from Mamba4Rec	`mamba4rec.py`, `tests/test_mamba4rec.py`
2	Delete fusion.py, llm_projection.py	remove files, fix imports
3	Single-phase training	`train.py`, `tests/test_train.py`
4	Add faiss-cpu dependency	`pyproject.toml`
5	Swap encoder to BGE	`llm_encoder.py`, `tests/test_llm_encoder.py`
6	Filter plots to movies/TV	`pipeline/filter_plots.py`, `tests/pipeline/test_filter_plots.py`
7	Encode plots + FAISS	`pipeline/encode_plots.py`, `tests/pipeline/test_encode_plots.py`
8	Content tower	`content_tower.py`, `tests/test_content_tower.py`
9	Update embedding store	`embedding_store.py`, `tests/test_embedding_store.py`
10	Reranker	`reranker.py`, `tests/test_reranker.py`
11	Graduation manager	`graduation.py`, `tests/test_graduation.py`
12	Chat provider	`chat_provider.py`, `tests/test_chat_provider.py`

Task	What	Creates/Modifies
13	Dual-arm inference	`inference.py`, `tests/test_inference.py`
14	Update configs	`config_ml32m.yaml`, remove `config.yaml`
15	Clean up deps	`pyproject.toml`, remove `main.py`
16	Integration tests	`tests/test_integration.py`
17	Lint + verify	all files

Dependency order: Tasks 1-2 must be sequential. Tasks 3-12 can be done in any order after Task 2. Task 13 depends on Tasks 5, 8, 9, 10. Tasks 14-15 can be done anytime. Task 16 depends on all prior tasks. Task 17 is last.